MBLP131 · Hafta 1 · Ders Notları

İşletim Sistemleri Temelleri

Genel Bakış ve Bilişimin Tarihi

Bilgisayar nedir? İşletim sistemi nedir? Ve bugüne nasıl geldik?

Öğretim Üyesi: Dr. Öğr. Üyesi Hasan Oğuz
Elektronik Haberleşme Teknolojisi Program Başkanı, Meslek Yüksekokulu
İstanbul Okan Üniversitesi

Ders Telegram kanalı: `[ders grubu bağlantısı]`

Duyurular, sorular ve laboratuvar desteği için kanala katılın.

Bu haftanın sonunda yapabilmeniz gerekenler

1. Bilgisayarı, *girdi* → *işlem* → *çıktı* modelini kullanarak komutları izleyen bir makine olarak tanımlamak.
2. İşletim sistemi (*operating system*, OS) için ilk, işe yarar bir tanım vermek ve işletim sisteminin yaptığı üç işi açıklamak.
3. İşletim sistemine *neden* ihtiyaç duyduğumuzu, işletim sistemli ve işletim sistemsiz bir bilgisayarı karşılaştırarak açıklamak.
4. Bilişim tarihinin önemli dönüm noktalarını, Babbage'ın Analitik Makine'sinden bulut bilişime kadar bir zaman çizelgesine yerleştirmek.
5. *Toplu işleme*, *çoklu programlama* ve *zaman paylaşımı* gibi fikirlerin neden icat edildiğini basit bir hesapla açıklamak.

İçindekiler

1 Bilgisayar Aslında Nedir? 2

2 İşletim Sistemi Nedir? 3

3 İşletim Sistemine Neden İhtiyaç Duyarız? 4

4 Bugüne Nasıl Geldik? İşletim Sistemlerinin Tarihi 4

4.1 0. Kuşak: Elektronikten önce (1837–1941) . . . 5

4.2 1. Kuşak: Vakum tüpleri ve işletim sisteminin yokluğu (1945–1955) . . . 7

4.3 GM-NAA I/O ve toplu işleme (1956) . . . 7

4.4 IBM System/360: tek işletim sistemi, çok sayıda makine (1964) . . . 8

4.5 Çoklu programlama: CPU'ya aynı anda birden çok iş (1960'lar) . . . 8
4.6 Zaman paylaşımı: aynı anda çok kullanıcı, tek makine (1960'lar) . . . 9
4.7 Multics: zamanında teslim edilemeyen dev (1965–1969) . . . 10
4.8 UNIX doğuyor (1969–1973) . . . 10
4.9 Kişisel bilgisayar (1977–1995) . . . 10
4.10 Linux doğuyor (1991) . . . 11
4.11 6. Kuşak: cebinizdeki bilgisayar (2007–2008) . . . 12
4.12 7. Kuşak: bulut (2006–günümüz) . . . 12
4.13 Zaman çizelgesinin tamamı, bir bakışta . . . 13

5 Bu Tarih Neden Önemli? 14

6 Özet 15

7 Terimler Sözlüğü 15

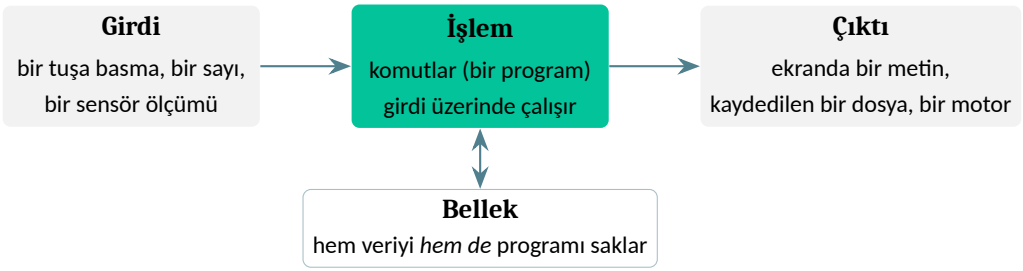
8 Kendiniz Deneyin 16

9 Tekrar Soruları 16

10 Sırada Ne Var: 2. Hafta 17

1 Bilgisayar Aslında Nedir?

Ekranı, klavyeyi ve uygulamaları bir kenara bırakın. Özünde **bilgisayar** (*computer*), *komutları izleyen* bir makinedir. Bir *girdi* alır, bu girdiyi bir komut listesine, yani bir **programa** (*program*) göre *işler* ve bir *çıktı* üretir.



Şekil 1: Girdi–işlem–çıktı (*input–process–output*, IPO) modeli. Bellek hem veriyi hem de programın kendisini tutar.

Şekil 1'deki iki ayrıntı dersin geri kalanı için önemlidir:

- **Program da tıpkı veri gibi bellekte saklanır.** Buna **depolanmış program** (*stored-program*) fikri denir. Bu sayede bilgisayarın ne yapacağını, hiçbir kabloyu değiştirmeden, yalnızca başka bir program yükleyerek değiştirebiliriz. Bölüm 4.2'de göreceğimiz gibi, ilk elektronik bilgisayarlar bu şekilde *çalışmıyordu*.

- **Döngü hiç durmaz.** İşlemci aynı çevrimi tekrar tekrar uygular: bellekten sıradaki komutu *getirir* (*fetch*), *çözer* (*decode*) ve *yürütür* (*execute*). Modern bir işlemci bunu saniyede milyarlarca kez yapar.

"Saniyede milyarlarca kez" ne kadar hızlıdır?

Saat frekansı $f = 3$ GHz olan bir işlemci bir saat çevrimini

$$T = \frac{1}{f} = \frac{1}{3 \times 10^{9}\ \text{s}^{-1}} \approx 0{,}33\ \text{ns}$$

sürede tamamlar. Bu sürede ışık yalnızca yaklaşık $c\,T \approx (3\times10^{8}\ \text{m/s})(0{,}33\times10^{-9}\ \text{s}) \approx 10\ \text{cm}$ yol alır. Dolayısıyla bir sabit diski yalnızca 10 ms beklemek bile yaklaşık 3×10^{7} saat çevrimine mal olur. Bu sayıyı aklınızda tutun: işletim sistemleri tarihinin büyük bölümü, bu çevrimleri *boşa harcamamakla* ilgilidir.

Karşılaşacağınız her bilgisayar, ister bir telefon, ister bir dizüstü bilgisayar, ister bir dronun uçuş kontrolcüsü, isterse bir çamaşır makinesi olsun, aynı döngüyü izler.

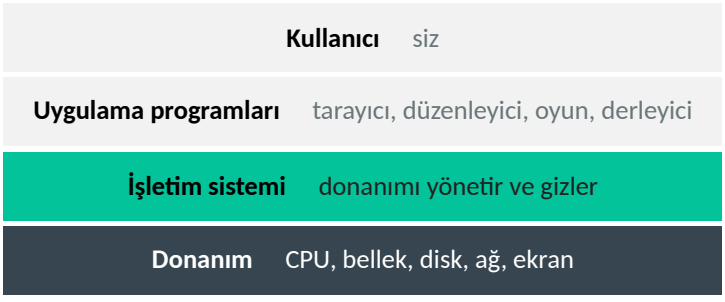
2 İşletim Sistemi Nedir?

İlk tanım (2. haftada netleştireceğiz)

İşletim sistemi, bilgisayarı açtığınızda ilk çalışan ve diğer programların çalışabilmesi için geri kalan her şeyi yöneten programdır.

Bu kısa tanımın içinde birbirinden ayrı üç görev saklıdır:

1. **İlk o başlar.** Güç düğmesine bastığınızda küçük bir ürün yazılımı (*firmware*) donanımı denetler ve ardından işletim sistemini yükler. Tarayıcınız, bir oyun ya da bir metin düzenleyici gibi diğer tüm programları işletim sistemi başlatır.
2. **Diğer her şeyi yönetir.** Birçok program işlemciyi, belleği, diski ve ağı aynı anda kullanmak ister. Kimin neyi, ne zaman alacağına işletim sistemi karar verir.
3. **Zor kısımları gizler.** *Kaydet*'e basarsınız; dosyanızın hangi diskin hangi kesimlerine (*sectors*) yazıldığını bilmeniz gerekmez. İşletim sistemi, karmaşık donanım ayrıntılarının yerine basit ve tek tip hizmetler sunar.



Şekil 2: İşletim sistemi, uygulama programları ile donanım arasında yer alır.

`Kaydet''e bastığınızda ne olur?

Bir metin düzenleyicide 2 MB'lık bir belgeyi kaydettiğinizi varsayalım.

1. Düzenleyici işletim sisteminden şunu ister: *"Bu baytları `rapor.docx` adlı bir dosyaya yaz."* Bu isteğe **sistem çağrısı** (*system call*) denir (2. haftada ilk kez, 3. haftada ayrıntılı olarak).
2. İşletim sistemi diskte boş yer bulur, dosyanın parçalarının nerede saklandığını kaydeder ve dosya adının doğru yeri göstermesi için klasörü günceller (dosya sistemleri, 7. hafta).
3. İşletim sistemi disk denetleyicisiyle onun kendi dilinde konuşur, işlemin bitmesini bekler ve düzenleyiciye "tamam" der.
4. Disk meşgulken işletim sistemi işlemciyi *başka* programların kullanımına verir (çoklu programlama, Bölüm 4.5).

Düzenleyicinin programcısı bütün bunlar için tek bir satır kod yazdı. Gerisini işletim sistemi yaptı.

3 İşletim Sistemine Neden İhtiyaç Duyarız?

İşletim sisteminin amacını görmenin en kolay yolu, işletim sistemi olmayan bir bilgisayarı hayal etmektir.

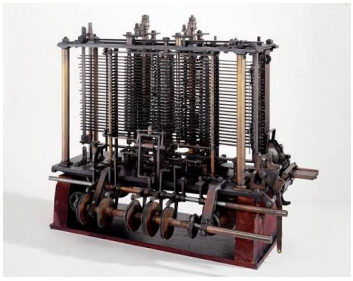
İşletim sistemi olmadan	İşletim sistemiyle
Bilgisayarı aynı anda yalnızca bir program kullanabilir.	Birçok program aynı donanımı güvenle paylaşır.
Her program diski, ekranı ve klavyeyi kendisi sürmek zorundadır; bunu doğru yapmak zor, bozmak ise kolaydır.	Programlar işletim sisteminden hizmet ister; donanımı bir kez ve doğru biçimde işletim sistemi yönetir.
Hatalı bir program başka bir programın belleğinin, hatta bütün makinenin üzerine yazabilir.	İşletim sistemi programları ayrı ve korumalı bellek alanlarında tutar.
Program giriş/çıkış (G/Ç) beklediği her an işlemci boşta kalır.	İşletim sistemi bekleme süresince işlemciyi başka bir programa verir.

Sol sütun hayal ürünü değildir: ilk bilgisayarların nasıl çalıştığını neredeyse birebir anlatır. Bu notların geri kalanı, sağ sütunun her seferinde bir sorun çözülerek nasıl inşa edildiğinin hikâyesidir.

4 Bugüne Nasıl Geldik? İşletim Sistemlerinin Tarihi

Bilişim tarihini *kuşaklara* ayırmak yaygın bir alışkanlıktır. Sınırlar yaklaşıktır; ancak her kuşak yeni bir donanım teknolojisi ve onunla birlikte yazılımın çözmesi gereken yeni bir sorun getirmiştir.

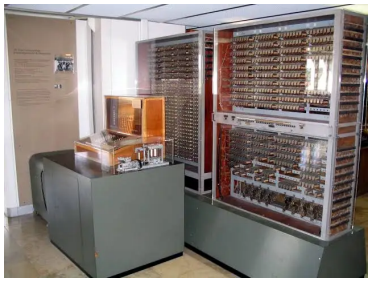
4.1 0. Kuşak: Elektronikten önce (1837--1941)



(a) Babbage'ın Analitik Makine için yaptığı "değirmen" deneme modeli



(b) Delikli kart okuyuculu Hollerith tablolama makinesi



(c) Konrad Zuse'nin Z3 makinesinin kopyası

Şekil 3: Elektronik bilgisayarlardan önceki programlanabilir makineler.

Charles Babbage ve Analitik Makine (1837)

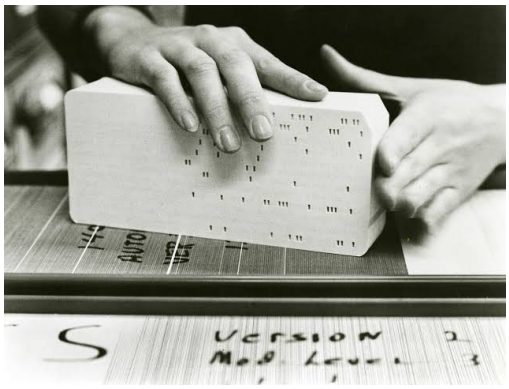
İngiliz matematikçi Charles Babbage, mekanik ve genel amaçlı bir bilgisayar olan **Analitik Makine**'yi (*Analytical Engine*) tasarladı. Tasarımında bugün her bilgisayarda bulunan iki parça zaten vardı: *değirmen* (*mill*) adını verdiği bir işlem birimi ve *depo* (*store*) adını verdiği bir bellek. Komutlar ve veriler, makineye delikli kartlarla verilecekti; bu fikir, kumaşa desen dokumak için delikli kartlar kullanan Jacquard dokuma tezgâhından alınmıştı. Makine hiçbir zaman tamamlanmadı; Şekil 3(a), değirmenin bir bölümüne ait deneme modelini göstermektedir.

Ada Lovelace 1843'te Analitik Makine üzerine, Bernoulli sayılarını hesaplamak için adım adım bir yöntem içeren notlar yayımladı. Bu yöntem, yaygın olarak yayımlanmış ilk bilgisayar programı kabul edilir. Lovelace, böyle bir makinenin yalnızca sayıları değil, her türlü simgeyi işleyebileceğini de fark etmişti.

Herman Hollerith ve delikli kart (1890)

1880 ABD nüfus sayımının elle işlenmesi yaklaşık sekiz yıl sürmüştü. **Herman Hollerith**, 1890 sayımı için elektromekanik bir **tablolama makinesi** (*tabulating machine*) yaptı. Her kişinin yanıtları bir karta açılan delikler olarak kaydediliyordu. Kart okuyucuya bastırıldığında yaylı iğneler deliklerden geçip cıva dolu küçük kaplara değiyor, bir elektrik devresini tamamlıyor ve bir sayaç kadranını bir birim ilerletiyordu. 1890 sayımı, öncekinin süresinin küçük bir kesrinde tamamlandı.

Hollerith'in şirketi daha sonra başka şirketlerle birleşti ve 1924'te **IBM** adını aldı. Delikli kartın kendisi ise programları ve verileri bilgisayara aktarmanın standart yolu olarak yaklaşık bir yüzyıl boyunca kullanıldı.



Şekil 4: Kart delme/doğrulama makinesinde okunan bir delikli kart. Her delik sütunu bir karakteri kodlar; bir kart genellikle bir program satırını taşırdı.

Tek bir kart ne kadar bilgi tutabilir?

Standart IBM kartında 80 sütun ve 12 satır olası delik konumu vardı. Her konumu bir ikili basamak (delik var / delik yok) olarak kullansaydık, bir kart en fazla

$$80 \times 12 = 960 \text{ bit} = 120 \text{ bayt}$$

bilgi tutabilirdi. Uygulamada her sütun bir karakteri kodluyordu; dolayısıyla bir kart 80 karakter taşıyordu. 2000 satırlık bir program bu yüzden 2000 kartlık, yaklaşık 35 cm yüksekliğinde bir deste gerektiriyordu ve bu destenin yere düşmesi gerçek bir felaketti.

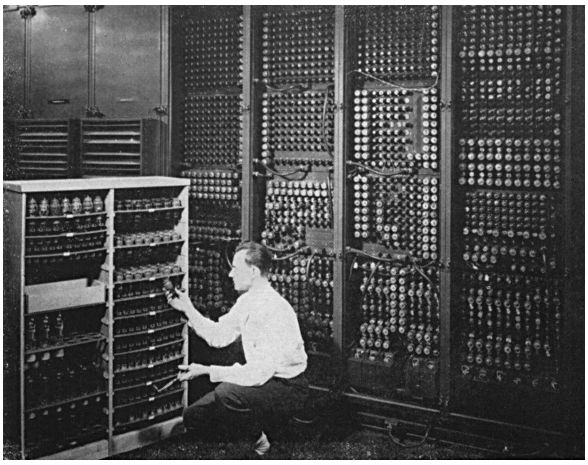
Konrad Zuse ve Z3 (1941)

Berlin'de mühendis **Konrad Zuse** bir dizi programlanabilir makine yaptı. 1941'de tamamlanan ve yaklaşık 2.600 telefon rölesinden oluşan **Z3**, çalışan ilk programlanabilir bilgisayarlardan biriydi. Programını delikli film şeridinden okuyordu. Özgün makine 1943'teki bir hava saldırısında yok oldu; Şekil 3(c)'deki makine 1960'larda yapılmış bir kopyadır.

0. Kuşaktan çıkarılacak ders

Bütün bileşenler zaten mevcuttu: bir işlemci, bir bellek, kartlar üzerinde programlar. Eksik olan *hızdı*; makineyi yönetecek hiçbir yazılım da yoktu.

4.2 1. Kuşak: Vakum tüpleri ve işletim sisteminin yokluğu (1945--1955)



Şekil 5: ENIAC (*Electronic Numerical Integrator and Computer* – Elektronik Sayısal Entegratör ve Bilgisayar), 1945.

ABD Ordusu için Pennsylvania Üniversitesi'nde 1945'te tamamlanan **ENIAC**, genel amaçlı ilk *elektronik* bilgisayarlardan biriydi. Bazı sayılar:

- yaklaşık 18.000 vakum tüpü, yaklaşık 30 (kısa) ton ağırlık ve yaklaşık 9 m $\times$ 15 m'lik bir odayı dolduran boyut;
- saniyede yaklaşık 5.000 toplama, yani masa hesap makinesi kullanan bir insandan binlerce kat hızlı;
- vakum tüpleri sık sık bozuluyor, teknisyenler arızalı tüpü bulup değiştirmek zorunda kalıyordu.

ENIAC'ın klavyesi, ekranı ve **işletim sistemi yoktu**. "Programlama", bağlantı panolarını fiziksel olarak yeniden kablolamak ve binlerce anahtarı ayarlamak demekti; bu da günler sürebiliyordu. ENIAC'ın ilk programcıları altı kadından oluşan bir ekipti: Kay McNulty, Betty Jennings, Betty Snyder, Marlyn Wescoff, Fran Bilas ve Ruth Lichterman.

Aynı anda yalnızca tek bir iş çalışıyor ve makinenin tüm denetimi o işte oluyordu. Yönetilecek hiçbir şey yoktu, çünkü başka bir yazılım yoktu.

4.3 GM-NAA I/O ve toplu işleme (1956)

1950'lerin ortasına gelindiğinde bilgisayarlar transistör kullanıyor, daha güvenilir hâle geliyor ve çok pahalıya mal oluyordu; dolayısıyla boşa geçen her dakika para demekti. Buna karşın her iş bittiğinde bir insan operatörün makineye gidip biten işi indirmesi, sıradaki işin teyplerini ve kartlarını takması ve işi başlatması gerekiyordu. Bütün bu süre boyunca bilgisayar hiçbir şey yapmıyordu.

1956'da General Motors Research ve North American Aviation, IBM 704 ana bilgisayarları (*mainframe*) için **GM-NAA I/O**'yu yazdı. Bu sistem bellekte **yerleşik monitör** (*resident monitor*) adı verilen küçük ve kalıcı bir program tutuyordu. Bu programın tek görevi, sıradaki işi otomatik olarak, birbiri ardına yükleyip çalıştırmaktı. GM-NAA I/O, *başka* programları yönetmek için yazılmış ve en yaygın atıf yapılan en erken program olduğundan çoğu zaman **ilk işletim sistemi** olarak anılır.

Bu bilişim biçimine **toplu işleme** (*batch processing*) denir:

1. **Kartları del.** Programcı işini bir deste delikli karta yazar.

2. **Toplu işi oluştur.** Bir operatör birçok işi toplar ve hepsini birlikte yükler.
3. **Otomatik çalıştır.** Monitör, işleri arada hiçbir insan müdahalesi olmadan birbiri ardına çalıştırır.
4. **Çıktıyı al.** Saatler, hatta bir gün sonra programcı kâğıt çıktısını alır ve bir yazım hatası yapıp yapmadığını ancak o zaman öğrenir.

Monitör kendi maliyetini neden çıkardı?

Her işin $t_{\text{çalışma}} = 2$ dakikalık hesaplamaya ihtiyaç duyduğunu ve bir insan operatörün iki iş arasında geçiş yapmak için $t_{\text{hazırlık}} = 8$ dakikaya ihtiyaç duyduğunu varsayalım. İşlemcinin faydalı iş yaptığı zamanın oranı, yani **kullanım oranı** (*utilization*),

$$U = \frac{t_{\text{çalışma}}}{t_{\text{çalışma}} + t_{\text{hazırlık}}} = \frac{2}{2 + 8} = 0{,}20 = \%20$$

olur. Yerleşik monitör geçiş süresini $t_{\text{hazırlık}} = 0{,}5$ dakikaya indirirse

$$U = \frac{2}{2 + 0{,}5} = 0{,}80 = \%80$$

elde edilir. Aynı makine, daha hızlı bir donanım olmadan, artık günde dört kat fazla iş yapar.

4.4 IBM System/360: tek işletim sistemi, çok sayıda makine (1964)

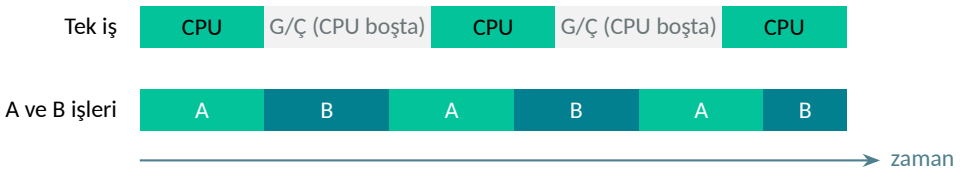
1964'ten önce her IBM bilgisayar modelinin kendine özgü ve birbiriyle uyumsuz bir komut kümesi vardı; bu nedenle daha büyük bir bilgisayar almak, bütün yazılımları yeniden yazmak anlamına geliyordu. IBM 1964'te **System/360**'ı duyurdu: küçükten çok büyüğe uzanan ve hepsi aynı programları çalıştıran bir bilgisayar ailesi. Bu ailenin işletim sistemi **OS/360**, o güne dek girişilmiş en büyük yazılım projelerinden biriydi ve gecikmesi ile bütçeyi aşmasıyla ün kazandı. Projenin yöneticisi Fred Brooks, bu projeden çıkardığı dersleri daha sonra *The Mythical Man-Month* (1975) adlı kitabında anlattı.

Temel fikir şudur: **işletim sistemi, tek bir makineye bağlı kalmadan birçok makineye taşınabilen, başlı başına bir ürün olabilir.**

4.5 Çoklu programlama: CPU'ya aynı anda birden çok iş (1960'lar)

Toplu işleme insanı döngüden çıkardı, ancak bu kez yeni bir israf ortaya çıktı. Tipik bir iş, zamanının önemli bir kısmını yavaş giriş/çıkış (G/Ç; *input/output*, I/O) işlemlerini, yani bir teybin, diskin ya da kartın okunmasını bekleyerek geçirir. Bir iş beklerken hızlı işlemci boşta kalır.

Çözüm **çoklu programlamadır** (*multiprogramming*): bellekte aynı anda *birkaç* iş tutmak. Çalışan iş G/Ç beklemeye başladığında işletim sistemi işlemciyi, hesaplama yapmaya hazır olan başka bir işe geçirir.



Şekil 6: Üstte: tek bir iş, G/Ç beklerken CPU'yu boşta bırakır. Altta: bellekte iki iş varken işletim sistemi bu boşlukları doldurur.

CPU kullanım oranı için basit bir model

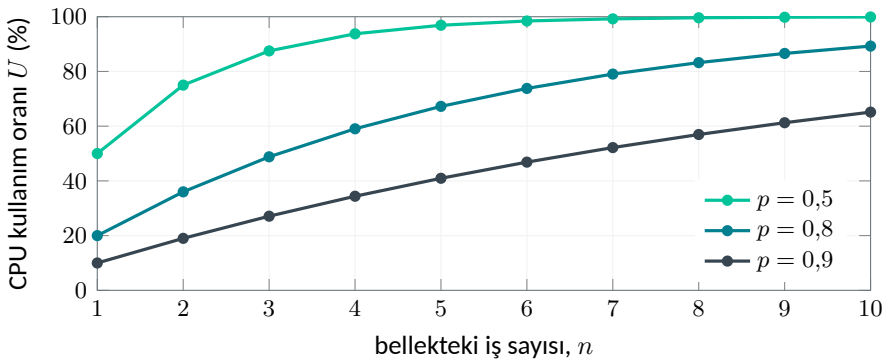
Her işin zamanının p kadarlık bir kesrini G/Ç bekleyerek geçirdiğini ve basitlik için işlerin birbirinden bağımsız olarak beklediğini varsayalım. CPU yalnızca bellekteki n işin *tamamı* aynı anda bekliyorsa boşta kalır; bunun olasılığı p^n'dir. Dolayısıyla

$$U(n) = 1 - p^n$$

olur. $p = 0{,}8$ olan, G/Ç ağırlıklı tipik bir iş için:

$$U(1) = 1 - 0{,}8 = \%20, \qquad U(4) = 1 - 0{,}8^4 \approx \%59, \qquad U(10) = 1 - 0{,}8^{10} \approx \%89.$$

Bellekte ne kadar çok iş varsa CPU o kadar meşgul ve verimlidir. Gerçek işler birbirinden bağımsız beklemediği için model basitleştirilmiştir; ancak çoklu programlamanın neden zahmetine değdiğini ve *belleğin* (6. hafta) neden bu kadar önem kazandığını gösterir: daha fazla bellek, bellekte aynı anda daha fazla iş tutulabilmesi demektir.



Şekil 7: Üç farklı G/Ç bekleme kesri p için $U(n) = 1 - p^n$.

4.6 Zaman paylaşımı: aynı anda çok kullanıcı, tek makine (1960'lar)

Çoklu programlama CPU'yu meşgul tuttu, ama programcı hâlâ işini teslim edip sonucu saatlerce bekliyordu. **Zaman paylaşımı** (*time-sharing*) bu sorunu, tek bir bilgisayara çok sayıda *terminal* (bir klavye ile bir yazıcı ya da ekran) bağlayarak çözdü. İşletim sistemi her kullanıcıya, hızlı bir sırayla, **zaman dilimi** (*time quantum*) adı verilen küçük bir CPU süresi verir. Her kullanıcı, makinenin tamamı kendisindeymiş gibi hisseder.

İlk başarılı uygulama, 1961'de MIT'de geliştirilen **CTSS**'tir (*Compatible Time-Sharing System*).

Neden anlık hissettirir?

$n = 20$ kullanıcının tek bir CPU'yu paylaştığını ve işletim sisteminin her birine $q = 10$ ms'lik bir zaman dilimi verdiğini varsayalım. En kötü durumda bir kullanıcı, yeniden çalışmadan önce diğer herkesi bekler:

$$T_{\text{bekleme}} \approx n\,q = 20 \times 10 \text{ ms} = 200 \text{ ms}.$$

Bir tuşa bastıktan sonra yaklaşık $0{,}2$ s'lik bir gecikme bir insan için neredeyse fark edilmez. CPU paylaşılır, ama her kullanıcı *etkileşimli* bir bilgisayar deneyimi yaşar. Öte yandan $n = 200$ kullanıcıda bekleme yaklaşık 2 s'ye çıkar ve bu can sıkıcıdır. q'yu iyi seçmek, 4.–5. haftalarda ele alacağımız zamanlama (*scheduling*) sorularından biridir.

Zaman paylaşımı, bugün kullanılan bütün çok kullanıcılı ve çok süreçli sistemlerin doğrudan atasıdır. İşlemcisini onlarca uygulama ve arka plan hizmeti arasında zaman paylaşımıyla bölüştüren telefonunuz da buna dâhildir.

4.7 Multics: zamanında teslim edilemeyen dev (1965--1969)

MIT, Bell Labs ve General Electric 1965'te **Multics** projesini başlattı: bir kamu hizmeti gibi bütün bir şehre hizmet verecek, sürekli açık, kolayca genişletilebilen ve güvenli bir zaman paylaşımlı sistem. Fikirleri çağının çok ötesindeydi: hiyerarşik dosyalar ve dizinler, dinamik bellek yönetimi ve ayrıntılı güvenlik denetimi. Ancak tasarım o kadar iddialıydı ki sistem planlanandan daha yavaş ve daha pahalı hâle geldi; Bell Labs 1969'da projeden çekildi. Multics sonunda çalıştı, fakat en kalıcı etkisi, projede çalışmış iki Bell Labs araştırmacısı üzerinde oldu.

4.8 UNIX doğuyor (1969--1973)

- **1969.** Ken Thompson, **UNIX**'in ilk sürümünü Bell Labs'te boşta duran bir PDP-7 mini bilgisayarında yazar. Sistem bilerek küçük ve basit tutulmuştur; Multics'in tam tersidir.
- **1970–1972.** Thompson ve Dennis Ritchie onu, her biri tek bir işi iyi yapan ve birbirine zincirlenebilen küçük ama güçlü araçlardan oluşan bir kümeye dönüştürür.
- **1972–1973.** Ritchie **C** programlama dilini tasarlar ve UNIX C ile yeniden yazılır. UNIX'in daha sonra neredeyse her donanımda çalışabilmesini sağlayan tek karar budur: sistemi yeni bir makineye taşımak için onu yeniden yazmak yerine büyük ölçüde yeniden derlemek yeterlidir.

> **UNIX felsefesi (Doug McIlroy'un özetiyle)**
> Tek bir iş yapan ve onu iyi yapan programlar yazın. Birlikte çalışacak programlar yazın. Metin akışlarını işleyen programlar yazın, çünkü metin evrensel bir arayüzdür.

Bu felsefeyi 9.–14. haftalarda doğrudan kullanacaksınız. Örneğin

```
cat access.log | grep "404" | wc -l
```

komutu, bir web sunucusu günlüğünün (*log*) kaç satırında 404 hata kodu geçtiğini saymak için üç küçük programı birbirine zincirler.

UNIX taşınabilir C diliyle yazıldığı ve yıllarca üniversitelerle paylaşıldığı için tek bir ürün olarak kalmadı; bütün bir aileye ayrıldı: **BSD** (erken internetin üzerinde çalıştığı TCP/IP ağ desteğini ekleyen Kaliforniya Üniversitesi, Berkeley), **HP-UX**, **AIX**, **SunOS/Solaris** ve daha sonra sertifikalı bir UNIX olan Apple'ın **macOS**'u.

4.9 Kişisel bilgisayar (1977--1995)

Apple II (1977)

Steve Wozniak tarafından tasarlanıp Apple Computer tarafından satılan **Apple II**, seri üretilen ilk başarılı kişisel bilgisayarlardan biriydi. Renkli grafik ve yerleşik bir BASIC diliyle geliyordu; bu sayede açıldığı anda programlanmaya hazırdı. İlk kez, bilgisayarı kullanan kişi ile bilgisayarın sahibi çoğu zaman aynı kişiydi.

IBM PC ve MS-DOS (1981)



Şekil 8: IBM Kişisel Bilgisayarı (*IBM Personal Computer*, model 5150), 1981.

IBM, 1981'de hazır parçalardan hızla bir araya getirilmiş **IBM PC** ile pazara girdi. Bilgisayarın işletim sistemi, mevcut bir sistemi (86-DOS) satın alıp uyarlayan Microsoft adlı küçük bir şirketten geldi; IBM onu PC DOS adıyla, Microsoft ise başka üreticilere **MS-DOS** adıyla sattı. MS-DOS, makine her açıldığında bir disketten yükleniyordu; sabit disk gerekmiyordu.

IBM, PC için ayrıntılı teknik belgeler yayımladı. Compaq'tan başlayarak başka şirketler, IBM'in sahip olduğu tek parçayı, yani BIOS adlı başlangıç ürün yazılımını yasal yollarla yeniden oluşturunca uyumlu "klon" makineler üretebildi. Binlerce şirket bunu yaptı ve "PC uyumlu" kavramı sonraki yirmi yıl boyunca sektör standardı oldu.

Grafik kullanıcı arayüzleri

- **Xerox PARC, 1973.** Araştırmacılar üst üste binen pencereler, simgeler ve fare içeren *Alto*'yu yapar. Alto büyük ölçüde bir araştırma makinesiydi ve hiçbir zaman yaygın olarak satılmadı.
- **Apple Macintosh, 1984.** Bu fikirleri kitlesel bir ürüne taşır: tıkla-seç kullanım, çöp kutusu simgesi, masaüstü benzetmesi ve komut yazma zorunluluğunun ortadan kalkması.
- **Microsoft Windows, 1985–1995.** MS-DOS'un üzerinde çalışan grafik bir kabuk olarak başlar; *Windows 95* ile çoğu insanın PC kullanma biçimi hâline gelir.

Çoğu kullanıcı için komut yazmak olağan olmaktan çıkar. Ancak komut satırı hiçbir zaman kaybolmaz: hâlâ yüzeyin altındadır ve 9.–14. haftalarda çalışacağınız yer de orasıdır.

4.10 Linux doğuyor (1991)

25 Ağustos 1991, bir Usenet iletisi

"I'm doing a (free) operating system (just a hobby, won't be big and professional like gnu) for 386(486) AT clones."
["386(486) AT klonları için (ücretsiz) bir işletim sistemi yapıyorum (sadece bir hobi, GNU gibi büyük ve profesyonel olmayacak)."] — Linus Torvalds, Helsinki'de 21 yaşında bir öğrenci

Linux yeni bir *çekirdektir* (*kernel*; bir işletim sisteminin özü). Sıfırdan yazılmıştır, ancak davranış ve komutlar bakımından bilerek UNIX ile uyumlu tutulmuştur. İçinde hiç UNIX kodu yoktur; "UNIX benzeri" (*UNIX-like*) bir sistemdir.

Yayılmasını iki etken sağladı:

- **Lisans.** Linux 1992'den itibaren **GNU Genel Kamu Lisansı** (*GNU General Public License*, GPL) ile yayımlandı: herkes kaynak kodunu okuyabilir, değiştirebilir ve yeniden dağıtabilir; ancak değiştirilmiş sürümler de açık kalmak zorundadır.
- **GNU.** Richard Stallman'ın başlattığı **GNU Projesi** 1983'ten beri, çekirdek dışındaki bütün standart UNIX araçlarının özgür sürümlerini yazıyordu. Eksik çekirdeği Linux sağladı. GNU ve Linux birlikte eksiksiz ve özgür bir işletim sistemi oluşturdu.

Bu arada sunucu odasında

Windows masaüstünü kazanırken UNIX ve Linux, çoğu insanın hiç görmediği makineleri kazandı. Bu sistemler en baştan çok sayıda eşzamanlı kullanıcı ve uzun, kesintisiz çalışma süresi için tasarlanmıştı; Linux'un lisans ücreti gerektirmemesi de sunucu çiftlikleri binlerce makineye ulaştığında belirleyici bir avantaj oldu. Bugün Linux, web sunucularının ve bulut sanal makinelerinin çoğunda ve 2017'den beri dünyanın en hızlı 500 süper bilgisayarının (TOP500 listesi) *tamamında* çalışmaktadır.

4.11 6. Kuşak: cebinizdeki bilgisayar (2007--2008)

- **iPhone (2007).** Apple'ın telefonu, basitleştirilmiş bir telefon işletim sistemi değil, masaüstü işletim sisteminin eksiksiz, uyarlanmış bir sürümünü (sonraki adıyla iOS) çalıştırıyordu. Parmakla kullanılan kapasitif çoklu dokunmatik ekranı, gerçek bir web tarayıcısı ve 2008'den itibaren bir App Store'u vardı. Modern, çok görevli bir işletim sisteminin pille çalışan bir cihazda çalışabileceğini kanıtladı.
- **Android (2008).** Android Inc. 2003'te kuruldu ve 2005'te Google tarafından satın alındı; ilk Android telefon (HTC Dream) 2008'de piyasaya çıktı. Android'in özü *uyarlanmış bir Linux çekirdeğidir* ve Google onu her telefon üreticisine ücretsiz lisanslar. Bunun sonucunda dünyadaki akıllı telefonların çoğu, uygulama simgelerinin altında Linux çalıştırmaktadır.

4.12 7. Kuşak: bulut (2006--günümüz)

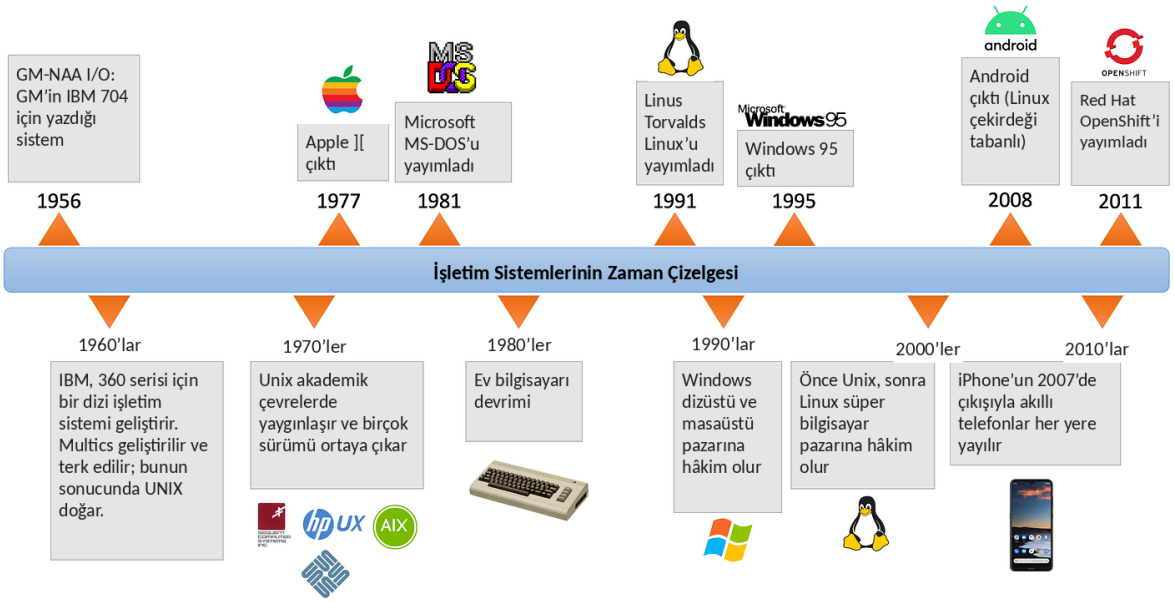
Sunucu donanımı giderek güçlendi, ancak çoğu program koca bir fiziksel makineyi tek başına kullanmaya ihtiyaç duymuyordu.

- **Sanallaştırma** (*virtualization*). *Hipervizör* (*hypervisor*) adı verilen bir program, tek bir fiziksel sunucunun, her biri kendi konuk işletim sistemini çalıştıran ve birbirinden yalıtılmış birkaç bağımsız bilgisayar (*sanal makine*) gibi davranmasını sağlar.
- **Bulut bilişim** (*cloud computing*). Amazon Web Services 2006'dan itibaren sanal makineleri saatlik olarak kiraladı: başkasının veri merkezini kendi bilgisayarınızmış gibi kullanırsınız.
- **Konteynerler** (*containers*). Daha da hafif bir fikir: konteyner, eksiksiz bir konuk işletim sistemi yerine ana makinenin Linux çekirdeğini paylaşır ve yalnızca uygulamayı ve uygulamanın ihtiyaç duyduklarını paketler. Red Hat *OpenShift* (2011) ve Google'ın *Kubernetes*'i (2014) gibi platformlar, binlerce konteyneri çok sayıda makine üzerinde çalıştırır ve yönetir.

Buradaki örüntüye dikkat edin: bir konteyner, özünde Linux çekirdeğinin süreçleri (*processes*) birbirinden yalıtan bir dizi özelliğidir. Bu dersin konuları olan süreçler, bellek koruması ve dosya sistemleri, modern bulutun üzerine inşa edildiği temellerdir.

4.13 Zaman çizelgesinin tamamı, bir bakışta

Yıl	Dönüm noktası	İşletim sistemleri açısından önemi
1837	Analitik Makine (Babbage)	İşlemci (“değirmen”) + bellek (“depo”) + kartlar üzerinde programlar
1843	Lovelace’ın notları	Yayımlanmış ilk bilgisayar programı
1890	Hollerith tablolama makinesi	Delikli kart bir yüzyıl boyunca girdi ortamı olur
1941	Zuse Z3	Çalışan ilk programlanabilir bilgisayarlardan biri
1945	ENIAC	Elektronik, ama işletim sistemi yok: tek iş, yeniden kablolamayla programlama
1956	GM-NAA I/O	Yerleşik monitör, toplu işleme: ilk işletim sistemi
1961	CTSS (MIT)	İlk zaman paylaşımlı sistem: etkileşimli bilişim
1964	IBM System/360, OS/360	Bütün bir makine ailesi için tek işletim sistemi
1965–69	Multics	İddialı zaman paylaşımı; UNIX’e ilham verir
1969	UNIX (Bell Labs)	Küçük, basit, birleştirilebilir araçlar
1973	UNIX’in C ile yeniden yazılması	Taşınabilir işletim sistemi: neredeyse her donanımda çalışır
1977	Apple II	Evler ve okullar için kişisel bilgisayar
1981	IBM PC, MS-DOS	PC uyumlu standart; disketten yüklenen işletim sistemi
1983	GNU Projesi	UNIX araçlarının özgür yazılım sürümleri
1984	Apple Macintosh	Kitlesel pazar için grafik arayüz
1991	Linux	Özgür, UNIX benzeri çekirdek; GNU ile eksiksiz, özgür bir işletim sistemi
1995	Windows 95	Grafik arayüz, PC kullanımının varsayılan biçimi olur
2006	Amazon Web Services	Bulut bilişim: sanal makineleri saatlik kiralamak
2007	iPhone	Telefonda eksiksiz, çok görevli işletim sistemi
2008	Android	Linux çekirdeği milyarlarca cebe girer
2011 / 2014	OpenShift / Kubernetes	Linux üzerinde büyük ölçekte yönetilen konteynerler



Şekil 9: GM-NAA I/O'dan (1956) OpenShift'e (2011) işletim sistemlerinin zaman çizelgesi.

5 Bu Tarih Neden Önemli?

1. **Her kuşak gerçek bir sorunu çözdü.** İşletim sistemi yok → otomasyon (toplu işleme) → CPU'yu paylaşmak (çoklu programlama) → CPU'yu etkileşimli olarak paylaşmak (zaman paylaşımı) → CPU'yu dünya çapında paylaşmak (bulut). Her işletim sistemi özelliği, onsuz birileri zaman ya da para kaybettiği için vardır.
2. **Eski fikirler hiçbir zaman gerçekten kaybolmaz.** Telefonunuz hâlâ çoklu programlama ve zaman paylaşımı yapar, arka planda hâlâ "toplu" işler (yedeklemeler, güncellemeler) çalıştırır.
3. **Bu ders de aynı sırayı izler.** Süreçleri, zamanlamayı, belleği ve dosyaları kabaca bu sorunları çözmek için icat edildikleri sırayla inceleyeceğiz.

Tarihsel fikir	MBLP131'de yeniden karşılaşacağınız yer
Sırasını bekleyen toplu işler	3. hafta: süreç kavramı, süreç durumları, süreç denetim bloğu (PCB)
Çoklu programlama ve zaman paylaşımı	4.–5. haftalar: CPU zamanlaması (temel kavramlar ve ölçütler; FCFS, SJF, Öncelik, Round Robin)
Multics'in bellek ve güvenlik fikirleri	6.–7. haftalar: bellek yönetimi, sayfalama, sanal bellek
Hiyerarşik dosyalar ve dizinler	7. hafta: dosya sistemleri ve dizin yapıları
UNIX'in küçük, birleştirilebilir araçları	9.–14. haftalar: Linux ve Bash, tamamen uygulamalı bölüm

6 Özet

Altı cümlede 1. hafta

1. Bilgisayar özünde komutları izler: girdi → işlem → çıktı; program da bellekte saklanır.
2. İlk çalışma tanımı: işletim sistemi, ilk başlayan ve diğer her şeyi yöneten programdır.
3. Programların donanımı güvenle paylaşabilmesi, programcıların ve kullanıcıların da donanımı kendilerinin yönetmek zorunda kalmaması için bir işletim sistemine ihtiyaç duyarız.
4. Tarih şu yolu izledi: yazılım yok (ENIAC) → toplu işleme → çoklu programlama/zaman paylaşımı → kişisel bilgisayarlar → mobil ve bulut.
5. Her adımı, boşta kalan işlemcinin maliyeti yönlendirdi; bu maliyeti $U = \frac{t_{\text{çalışma}}}{t_{\text{çalışma}} + t_{\text{hazırlık}}}$ ve $U(n) = 1 - p^n$ gibi basit formüllerle kestirebiliriz.
6. Akılda tutulmaya değer iki tarih: UNIX, 1969; dönemin ilerleyen haftalarında uygulamalı olarak kullanacağımız sistem olan Linux, 1991.

7 Terimler Sözlüğü

Çekirdek (kernel)
İşletim sisteminin her zaman bellekte duran, donanımı denetleyen ve kaynakları yöneten temel parçası.

Çoklu programlama (multiprogramming)
Bir iş G/Ç beklerken CPU'nun başka bir işe geçebilmesi için bellekte aynı anda birkaç iş tutmak.

Depolanmış program bilgisayarı (stored-program computer)
Programını veri gibi bellekte saklayan ve bu nedenle yeniden kablolama yapılmadan programı değiştirilebilen bilgisayar.

İşletim sistemi (operating system, OS)
İlk başlayan ve bilgisayarın donanımını ve diğer bütün programları yöneten program.

Kullanım oranı (utilization)
Bir kaynağın (genellikle CPU'nun) faydalı iş yaparak geçirdiği zamanın oranı.

Program
Bilgisayara ne yapacağını söyleyen komutlar listesi.

Sanal makine (virtual machine)
Gerçek (fiziksel) bir makinenin üzerinde, kendi işletim sistemini çalıştıran, yazılımla oluşturulmuş bilgisayar.

Sistem çağrısı (system call)
Bir programın işletim sisteminden bir hizmet istemesi (örneğin "bu dosyayı aç").

Toplu işleme (batch processing)
Bir araya toplanmış bir grup işi, aralarında hiçbir insan etkileşimi olmadan otomatik olarak birbiri ardına çalıştırmak.

Yerleşik monitör (resident monitor)
Bellekte kalıcı olarak tutulan ve işleri otomatik olarak yükleyip başlatan küçük program; işletim sisteminin

en erken biçimi.

Zaman dilimi (time quantum)
İşletim sisteminin her programa ya da kullanıcıya sırayla verdiği küçük CPU süresi.

Zaman paylaşımı (time-sharing)
CPU'yu çok sayıda etkileşimli kullanıcı arasında, her biri makinenin tamamı kendisindeymiş gibi hissedecek kadar hızlı biçimde değiştirmek.

8 Kendiniz Deneyin

Çoklu programlamayı kendi bilgisayarınızda görün

Windows: Görev Yöneticisi'ni (*Task Manager*) açmak için `Ctrl+Shift+Esc` tuşlarına basın ve *İşlemler* (*Processes*) sekmesine bakın. **Linux/macOS:** bir terminal açın ve `top` yazın (çıkmak için `q` tuşuna basın).

1. Şu anda kaç süreç çalışıyor? (Genellikle 100'den fazla.)
2. Bilgisayarınızda kaç CPU çekirdeği var? (Görev Yöneticisi → *Performans*.)
3. Süreç sayısı çekirdek sayısından çok daha fazlaysa bu süreçlerin hepsi nasıl "çalışıyor" olabilir? *Zaman paylaşımı* ve *çoklu programlama* kavramlarını kullanarak iki cümle yazın.

9 Tekrar Soruları

Kavramsal

1. Girdi–işlem–çıktı modelini çizin ve bir akıllı telefon kamerası için her bölüme bir örnek verin.
2. İşletim sisteminin ilk çalışma tanımını yazın ve üç görevini kendi cümlelerinizle açıklayın.
3. İşletim sistemi yokken birkaç programın aynı bilgisayarı kullanmak istemesi durumunda ortaya çıkan iki sorunu belirtin.
4. ENIAC neden "hiç işletim sistemi olmayan" bir bilgisayar olarak tanımlanır?
5. *Yerleşik monitör* neydi ve hangi sorunu çözdü?
6. *Çoklu programlama* ile *zaman paylaşımı* arasındaki farkı açıklayın. Bunlardan hangisi esas olarak CPU verimliliğiyle, hangisi esas olarak kullanıcı deneyimiyle ilgilidir?
7. UNIX'in 1973'te C ile yeniden yazılması neden bu kadar önemli bir karardı?
8. Linux'ta hiç UNIX kodu yoktur. Hangi anlamda yine de UNIX ailesinin bir üyesidir?
9. Windows masaüstünü kazanmış olsa da Linux'un sunucularda kazanmasının bir nedenini belirtin.
10. Bir konteyner sanal makineden nasıl ayrılır?

Sayısal

11. Bir iş $t_{\text{çalışma}} = 3$ dk çalışıyor ve operatörün işler arasında $t_{\text{hazırlık}} = 9$ dk'ya ihtiyacı var. CPU kullanım oranını hesaplayın. $U = \%90$ olması için $t_{\text{hazırlık}}$ ne olmalıdır?
12. İşler zamanlarının $p = 0{,}7$'sini G/Ç bekleyerek geçiriyor. $U(n) = 1 - p^n$ formülünü kullanarak $n = 1, 2, 5$ için U'yu bulun. $U \geq \%95$ sağlayan en küçük n nedir?
13. Zaman paylaşımlı bir sistemde n kullanıcı ve $q = 20$ ms'lik bir zaman dilimi var. Kullanıcılar $0{,}5$ s'den uzun beklerlerse şikâyet ediyor. Sistemin destekleyebileceği en fazla kullanıcı sayısı nedir?
14. Bir işlemci $2{,}5$ GHz'te çalışıyor. Bir disk için 5 ms beklerken kaç saat çevrimi geçer?

Sayısal soruların yanıtları

11. $U = \dfrac{3}{3+9} = \%25$. $U = 0{,}9$ için: $\dfrac{3}{3 + t_{\text{hazırlık}}} = 0{,}9 \Rightarrow t_{\text{hazırlık}} = \dfrac{3}{0{,}9} - 3 \approx 0{,}33$ dk $= 20$ s.
12. $U(1) = \%30$, $U(2) = 1 - 0{,}49 = \%51$, $U(5) = 1 - 0{,}7^5 \approx \%83$. $0{,}7^n \leq 0{,}05$ olmalıdır, yani $n \geq \dfrac{\ln 0{,}05}{\ln 0{,}7} \approx 8{,}4$; dolayısıyla $n = 9$.
13. $n\,q \leq 0{,}5$ s $\Rightarrow n \leq \dfrac{0{,}5}{0{,}020} = 25$ kullanıcı. (Gerçek sistemler kullanıcılar arasında geçiş yaparken de zaman kaybeder; bu nedenle gerçek sınır biraz daha düşüktür.)
14. $N = f\,t = (2{,}5 \times 10^9\ \text{s}^{-1})(5 \times 10^{-3}\ \text{s}) = 1{,}25 \times 10^7$ çevrim.

10 Sırada Ne Var: 2. Hafta

Gelecek hafta gayriresmî tanımımızın yerine kesin bir tanım koyacak ve işletim sisteminin içine bakacağız:

- işletim sisteminin biçimsel tanımı ve iki klasik bakış açısı: *kaynak yöneticisi* (*resource manager*) ve *genişletilmiş makine* (*extended machine*);
- işletim sisteminin tasarım hedefleri: kolaylık, verimlilik ve gelişebilirlik;
- bir bilgisayar sisteminin dört bileşeni: donanım, işletim sistemi, uygulama programları ve kullanıcılar;
- işletim sisteminin donanımla nasıl çalıştığı: kesmeler (*interrupts*), bellek hiyerarşisi, çift kip (*dual mode*) ve C ile `strace` kullanarak sistem çağrılarına ilk bakış;
- işletim sistemlerinin nasıl inşa edildiği: monolitik, katmanlı, mikroçekirdek ve modüler yapılar.

İleri okuma (isteğe bağlı)

- R. H. Arpaci-Dusseau ve A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, ücretsiz çevrim içi ders kitabı: `https://pages.cs.wisc.edu/~remzi/OSTEP/`
- A. Silberschatz, P. B. Galvin ve G. Gagne, *Operating System Concepts*, 10. baskı, Wiley, 1. bölüm.
- A. S. Tanenbaum ve H. Bos, *Modern Operating Systems*, 4. baskı, Pearson, Bölüm 1.2 ("History of Operating Systems").
- D. M. Ritchie, "The Evolution of the Unix Time-sharing System" (1979): `https://www.bell-labs.com/usr/dmr/www/hist.html`